

夯智交互 Python 后端实习生面试问答

马素超面试备考手册

本手册用于准备上海夯智交互文化科技的 Python 后端实习生面试。岗位把 Python 服务开发、数据库与异步任务、测试 Debug、云端交付、LLM 应用和 Coding Harness 放在同一条交付链上。你的回答主线应是：我能把需求落成可维护的服务，会验证 AI 生成代码，并对运行结果负责。

先练这八个问题

1. 为什么从以 Java 为主的 homegrown 项目转向 Python 后端岗位。
2. 慧学平台中 Flask、SQLAlchemy、Celery、Redis 和 PostgreSQL 怎样组成一条完整业务链。
3. FastAPI 中 `async def` 何时真正提升吞吐，何时反而阻塞事件循环。
4. Celery 任务至少一次投递时怎样避免重复计算和重复写入。
5. 你怎样用测试、静态检查和运行证据验证 Coding Agent 生成的代码。
6. Structured Output 与普通 JSON 提示有什么区别，解析失败怎么处理。
7. Tool Calling 为什么必须在执行前做权限、参数和业务规则校验。
8. 文献 RAG 的准确率提升约 23% 和 Token 减少 21% 怎样计算。

面试官可能怎样评估你

从 JD 可以判断，面试不会只考框架 API。面试官更可能沿着一个真实改动追问需求拆解、数据模型、异步边界、测试覆盖、故障定位和部署验证，再检查你能否审阅 AI 生成结果。简历中的 Python 平台与 Agent 实习是主要证据，homegrown 用于补充复杂状态、模型适配和完整交付能力。

使用方式

每题先用 60 至 90 秒口述参考回答，再练追问。P0 为优先题，P1 为常见深入题。凡是个人职责、上线规模、性能数字和工具熟练度，都要以你能展示的代码、记录或截图为准。文中的改进方案不能当成已经上线的事实。

阅读导航

第 2 至 4 页：岗位判断、匹配度与自我介绍。第 5 至 17 页：Python、FastAPI、数据、测试、稳定性和云端交付。第 18 至 22 页：Coding Harness、LLM、Agent、RAG 与评测。第 23 至 25 页：实习和项目深挖。第 26 至 28 页：现场题、系统设计、模拟面试与准备清单。第 29 页：资料来源。

编写日期 2026 年 9 月 9 日。面试关注点来自招聘页面信息与简历内容的结合分析，不代表公司真实题库。

01 岗位信号与面试官视角

01 P0 为什么想应聘这个岗位

考察点：动机是否对应岗位实际工作，而不是泛泛表达对 AI 的兴趣。

参考回答：这个岗位吸引我的地方是，它要求把 Python 后端、异步任务和 AI 应用一起交付，并明确强调代码质量、系统稳定性和最终结果。我在慧学平台做过 Flask、SQLAlchemy、Celery、Redis 和 PostgreSQL，在 Agent 实习中接触过 FastAPI、LangGraph、Tool Calling 和失败重试。我的 Java 项目还训练了状态机、模型适配和 SSE 等工程能力。我希望继续做 AI 应用后端，同时把 Python 工程化补得更扎实。

追问：为什么是这家公司？答：目前我能确认的是岗位目标和团队的工程取向。我会说明这些要求与自身经历的具体交集，并向面试官了解产品场景、当前服务规模和实习生负责的交付边界，不臆测没有公开说明的业务。

02 P0 你认为面试官最关心什么

考察点：能否从 JD 提炼真正的交付标准。

参考回答：我判断有三层。第一层是 Python 基础和 Web 服务能力，能独立写清晰的 API、数据库和异步代码。第二层是可靠性，遇到超时、重复任务、连接池耗尽或第三方失败时能定位和恢复。第三层是 AI 辅助开发能力，能让 Claude Code、Codex 或 Cursor 提高速度，同时用测试、类型检查、日志和代码审查验证结果。面试官可能会拿一个项目细节连续追问，观察我是否真的理解代码。

追问：怎样证明不是只会拼框架？答：从一个请求的入口讲到事务、任务队列、缓存、日志和测试，并解释一个失败路径；能写出最小代码或 SQL，比罗列技术名词更有说服力。

03 P0 你怎样理解 Coding Harness

考察点：是否理解岗位中这一特殊关键词。

参考回答：我把 Coding Harness 理解为围绕编码 Agent 建立的工作环境和约束，不等于某个聊天工具。它应把仓库说明、任务边界、可用命令、测试入口、格式检查、日志和变更审查组织起来，让 Agent 能获取必要上下文并产生可验证的增量。好的 Harness 还要限制敏感操作、缩小改动范围、保存执行证据，并在人类确认后完成高风险步骤。

追问：规则写得越多越好吗？答：不是。上下文过长会稀释关键约束。应把稳定规则放在仓库级说明，把任务信息放在当前提示，把机械约束交给 lint、测试、hook 或权限系统。

02 简历匹配度与回答边界

04 P0 你的简历为什么更像 Java 候选人

考察点：是否能正面解释语言主线差异。

参考回答：我的个人项目 homegrown 目前以 Java 为主，但我并不是从零转 Python。慧学平台使用 Flask RESTX、SQLAlchemy、Celery 和 Redis，我的 Agent 实习也使用 Python、FastAPI 与 LangGraph。Java 项目让我积累了状态建模、鉴权、模型兼容和流式交付经验，这些设计能力可以迁移；Python 岗位需要我重点证明的是语言习惯、异步边界、类型检查和测试工具链，而不是把 Java 写法照搬过来。

追问：为什么不继续投 Java？答：我的目标是 AI 应用后端。Python 在 Agent 与模型生态里更接近主流工具链，而我已有可解释的 Python 项目基础，愿意用实际代码证明转换成本可控。

05 P0 你与岗位最匹配的三条证据是什么

考察点：能否选择证据并建立因果关系。

参考回答：第一，慧学平台覆盖 Flask API、SQLAlchemy、独立 PostgreSQL 库、Celery 队列和 Redis，能说明我做过完整后端链路。第二，Agent 实习中做过自然语言任务到扫描工具的映射、LangGraph 路由、失败重试和记忆管理，对应 Tool Calling 与 Agent。第三，我在 homegrown 中处理过模型 Provider 适配、结构化输出和 SSE，能说明我会处理模型接口的不确定性。面试时我会为每条准备入口代码、运行截图和一个故障案例。

追问：哪一条最容易被质疑？答：实习和团队项目容易被追问个人贡献，需要明确自己写过的模块、共同完成的部分和只能描述接口的部分。

06 P0 你的明显缺口是什么

考察点：是否诚实，并有短周期补齐路径。

参考回答：JD 加分项中的 AWS、EC2、ECS、RDS，以及 uv、Ruff、Pydantic v2，没有作为我简历里的主要生产经验。我不会把了解过包装成落地经验。面试前我会用一个最小 FastAPI 服务展示 Pydantic v2、pytest、Ruff、uv、Docker 和 CI，补齐可验证的工程链；云平台部分先讲清容器、配置、日志、健康检查和数据库迁移这些通用原则，再如实说明具体云资源的实践程度。

追问：如果岗位必须有 AWS 经验呢？答：先确认是硬要求还是加分项；如果确实要求独立维护生产资源，我会承认当前经验不足，不用概念性回答冒充操作经验。

面试前必须核对

确认慧学平台和实习项目中自己实际提交过的文件、框架版本、部署方式和可展示材料。确认文献 RAG 指标的样本数、评测规则与基线。确认到岗日期、每周出勤天数和可持续月份。任何数字如果没有记录，应删除或改为定性描述。

03 自我介绍与项目主线

07 P0 请做一分钟自我介绍

考察点：能否快速建立与 Python 后端和 AI 工程的关联。

参考回答：您好，我叫马素超，是上海理工大学电子信息专业 2027 届硕士，本科是软件工程。我的方向是后端和 Agent 应用开发。Python 方面，我在慧学教育平台中使用 Flask RESTX、SQLAlchemy、Celery、Redis 和 PostgreSQL，做过多租户部署、异步画像计算、鉴权及可观测性；在 Agent 实习中使用 FastAPI、LangGraph 和 LangChain，参与工具调用、任务路由、记忆与失败重试。最近的 homegrown 项目以 Java 为主，但让我深入处理了状态机、结构化输出、SSE 和多模型适配。我希望应聘这个岗位，把这些经验用于可测试、可维护的 Python AI 服务。

追问：为什么介绍 Java 项目？**答：**只把它作为复杂工程能力的补充，随后立即回到 Python 项目和岗位要求，不让面试主线偏离。

08 P0 哪个项目最能证明你能做这个岗位

考察点：项目选择是否匹配 JD。

参考回答：我会优先讲慧学平台，因为它同时包含 Flask 后端、SQLAlchemy、PostgreSQL、Celery、Redis、Docker Compose、Gunicorn 和监控链路。一个典型流程是业务更新后标记画像数据为脏，后台任务按需增量重算，结果再供查询和推荐使用。这个流程可以展开数据一致性、任务幂等、队列隔离、连接池和故障观测。Agent 实习则作为第二条线，重点回答 FastAPI、LangGraph、Tool Calling 和记忆。

追问：能现场演示什么？**答：**只承诺确实可访问的页面、可脱敏代码、日志或架构图；提前验证账号、网络与数据，不把临时不可用解释成偶然。

09 P1 入职前两周你能交付什么

考察点：是否能形成小步、可验证的交付计划。

参考回答：我会先跑通仓库、测试和本地依赖，确认代码规范、发布流程及事故边界。然后选择一个范围明确的 API 或缺陷，补齐回归测试后完成修改，用日志和接口结果验证。涉及 Agent 时，会先理解 Prompt、工具 Schema、失败处理和评测样例，再改行为。两周内的目标不是大改架构，而是完成一个可审查的小闭环，并把发现的问题和后续建议写清楚。

追问：如果文档很少怎么办？**答：**从启动脚本、依赖文件、CI、路由入口和测试反推系统；把已确认事实与待确认问题分开记录，再找维护者校准。

04 Python 对象与资源管理

10 P0 Python 可变默认参数为什么危险

考察点：是否理解函数定义时求值与对象复用。

参考回答：默认参数在函数定义时创建，不是每次调用都重新创建。如果默认值是列表或字典，多次调用会共享同一对象，导致状态串联。安全写法通常把默认值设为 `None`，在函数内创建新对象。测试要连续调用两次，验证第二次不继承第一次的数据。对 `Pydantic` 模型或 `dataclass` 也要使用各自的 `default factory`。

追问：什么时候故意共享默认对象？答：很少，而且意图不直观。若需要缓存或共享状态，应使用显式的模块对象、缓存装饰器或依赖容器。

11 P1 生成器和迭代器有什么价值

考察点：能否把语言特性连接到后端场景。

参考回答：迭代器提供逐项读取协议，生成器用 `yield` 更简洁地保存执行状态。它们适合流式处理大文件、分页拉取和逐块生成响应，避免一次性加载全部数据。但惰性执行也会延后异常和资源释放，所以文件、游标或网络流要有清晰的生命周期，客户端断开时还要停止生产。

追问：生成器一定省内存吗？答：只有消费链路也保持流式时才省。若最后调用 `list` 或在中间积累全部结果，内存优势会消失。

12 P1 上下文管理器怎样保证资源释放

考察点：是否理解 `with` 和异常路径。

参考回答：上下文管理器把获取与释放资源放在 `enter` 和 `exit` 协议里，`with` 块无论正常结束还是抛异常都执行退出逻辑。数据库会话、文件、锁和追踪 `span` 都适合这种模式。异步资源使用 `async with`。实际代码还要区分提交、回滚和关闭，不能把 `close` 当成事务成功。

追问：在 `FastAPI` 中数据库会话怎么管理？答：常用 `yield` 依赖或 `lifespan` 管理，创建后注入请求，`finally` 中关闭；事务提交与回滚由服务边界明确控制。

05 Python 数据结构与并发选择

13 P0 list tuple dict set 怎样选

考察点：基础是否扎实，是否理解复杂度和语义。

参考回答：list 适合有序可变序列；tuple 表达固定记录或可哈希组合；dict 做键值查找并保留插入顺序；set 用于去重和成员测试。平均情况下 dict 和 set 查找接近 $O(1)$ ，list 中间插入或成员扫描通常是 $O(n)$ 。选择时先看语义，再看访问模式，不能只背复杂度，因为哈希质量、内存占用和数据规模也会影响结果。

追问：为什么数据库结果不直接全放 set？答：set 会丢失重复语义且占额外内存，排序也不保留。应该在 SQL 层完成可下推的过滤和去重。

14 P1 浅拷贝和深拷贝有什么区别

考察点：能否识别嵌套可变对象的共享。

参考回答：浅拷贝创建外层容器，但内部对象仍共享引用；深拷贝递归复制。配置、请求模板或测试夹具包含嵌套列表和字典时，浅拷贝后修改内部值可能污染原对象。深拷贝并非默认答案，它成本更高，也可能错误复制连接或锁。更好的办法是使用不可变结构、明确构造新对象，或只复制需要修改的层级。

追问：怎样写测试？答：修改副本的嵌套字段，断言原对象不变，并覆盖自定义对象的复制行为。

15 P0 线程 进程和 asyncio 怎么选

考察点：能否根据负载而不是习惯选并发模型。

参考回答：网络和数据库等待多的 I/O 任务适合 asyncio 或线程；CPU 密集任务需要多进程、原生扩展或独立计算服务。CPython 的 GIL 会限制同一进程内多个 Python 线程并行执行纯 Python CPU 代码，但线程仍能覆盖 I/O 等待。FastAPI 的异步接口适合异步客户端链路，图片处理或重计算不应直接压在事件循环上。

追问：Celery 属于哪类？答：它是分布式任务队列，可以把耗时或需重试的任务移出请求进程；Worker 内部仍需按任务类型选择进程、线程或其他执行池。

06 类型系统与 Pydantic

16 P0 Python 类型注解会在运行时强制校验吗

考察点：是否区分静态分析与运行行为。

参考回答：普通类型注解主要服务于阅读、IDE 和 mypy、Pyright 等静态检查，解释器通常不会自动拒绝错误类型。Pydantic 会在模型创建或显式验证时进行运行时解析与校验。因此边界数据应通过 Schema 校验，内部代码通过类型注解和静态检查减少错误，不能因为写了 str 或 int 就假设外部 JSON 已安全。

追问：Any 有什么风险？答：它会绕过大量静态检查并把错误推迟到运行时。第三方边界可暂时使用，但应尽快收窄到 Protocol、TypedDict 或具体模型。

17 P1 Protocol 和继承接口有什么区别

考察点：是否理解结构化类型。

参考回答：Protocol 支持结构化子类型，对象只要具有所需方法和属性就满足约束，不必显式继承。这适合为 LLM Client、存储或通知服务定义小接口，便于替换实现和测试。运行时依赖注入仍要提供具体对象；如果需要 isinstance 检查，要谨慎使用 runtime checkable，并知道它只检查结构的一部分。

追问：为什么接口要小？答：小接口降低耦合，测试替身更简单，也减少 Coding Agent 修改无关模块的机会。

18 P0 Pydantic v2 在 API 中承担什么角色

考察点：是否会把 Schema 当作边界契约。

参考回答：Pydantic 模型定义字段、约束、嵌套结构和序列化形式，可用于请求、响应及工具参数。外部输入默认可能发生类型转换，金额、权限或模型工具参数等敏感边界应考虑严格模式和 extra forbid，避免悄悄接受未知字段。模型验证通过只说明结构符合约束，不能代替业务校验，例如订单归属、余额和状态流转仍需服务层判断。

追问：model construct 能否用于加速？答：它跳过验证，只适合已经可信且经过校验的数据。未经验证的外部输入不能使用。

07 Asyncio 与 FastAPI 异步边界

19 P0 `async def` 为什么不等于自动并发

考察点：是否真正理解事件循环。

参考回答：调用 `async def` 只会得到协程对象，必须被 `await` 或创建为 `Task` 才执行。并发来自任务在等待可挂起的 I/O 时把控制权交还事件循环。如果协程内部调用同步 HTTP、同步数据库或长时间 CPU 计算，事件循环仍会被阻塞。排查时要确认依赖库是否原生异步，并通过延迟分布和事件循环阻塞监控验证。

追问：什么时候在 FastAPI 里写普通 `def`？答：当调用的是阻塞库且暂时不能替换时，可用普通 `def` 让框架在线程池执行，但还要限制并发，避免线程和连接被耗尽。

20 P0 怎样给第三方 LLM 调用设置超时和并发上限

考察点：是否能防止外部依赖拖垮服务。

参考回答：我会设置连接、读取和总 `deadline`，使用 `Semaphore` 或限流器限制每个 `Provider` 的并发，并让超时向下游取消传播。重试只针对明确可恢复错误，采用指数退避和随机抖动，同时受总 `deadline` 与次数限制。请求还要有幂等标识和 `trace id`。若服务已经过载，应快速返回可解释错误或进入队列，而不是无限等待。

追问：取消任务后 `finally` 会执行吗？答：通常会在取消异常传播时执行，所以清理应放在 `finally`。不要吞掉 `CancelledError`，否则结构化并发和超时可能失效。

21 P1 `asyncio gather` 和 `TaskGroup` 怎样选

考察点：是否理解并发任务的失败语义。

参考回答：`gather` 适合收集一组已知 `awaitable` 的结果，但要明确某个失败时其他任务是否继续。`TaskGroup` 提供更清晰的结构化并发边界，一个任务发生未处理异常时会取消其余任务并在退出时汇总异常。无论选择哪种，都要限制任务数量、处理取消并保证资源释放。批量调用模型时还要避免一次创建成千上万个 `Task`。

追问：`return exceptions` 等于容错吗？答：不是。它只是把异常作为结果返回，调用方仍要分类、记录并决定部分成功是否可接受。

08 FastAPI Flask 与 Django

22 P0 FastAPI Flask Django 怎样选

考察点：是否根据项目约束选择框架。

参考回答：FastAPI 适合类型驱动的 API、异步 I/O 和自动 OpenAPI；Flask 核心轻量，现有生态和团队经验成熟时可以快速扩展；Django 自带 ORM、管理后台和认证，更适合完整业务系统。选择要看现有代码、团队熟悉度、同步或异步依赖、后台需求和迁移成本。慧学平台使用 Flask 并不意味着应该为了新框架重写，除非收益能覆盖风险。

追问：FastAPI 一定更快吗？答：框架开销往往不是主要瓶颈。数据库、外部 API、序列化、Worker 配置和阻塞调用更关键，需要压测而不是凭印象判断。

23 P1 FastAPI 依赖注入和 lifespan 怎样使用

考察点：资源生命周期是否清楚。

参考回答：依赖注入适合认证用户、数据库会话、配置和权限检查，可以形成可测试的请求级边界。lifespan 适合应用启动时加载共享客户端或模型，关闭时释放连接。请求级对象不能误放成全局可变状态，全局客户端则应确认线程安全与连接池配置。测试时可以覆盖依赖，并验证启动与关闭钩子。

追问：yield 依赖抛异常怎么办？答：finally 仍要关闭资源；事务层根据异常回滚，异常映射统一转换为稳定的 API 错误。

24 P1 怎样组织一个可维护的 FastAPI 项目

考察点：是否能控制边界而非堆路由文件。

参考回答：路由负责协议解析和响应，服务层表达用例与事务，Repository 隔离持久化，外部 Client 封装第三方协议，Schema 与领域对象按需要分开。配置、日志、异常映射和依赖装配集中管理。模块应围绕业务能力划分，测试可以从服务层开始，再覆盖 API 契约。目录结构只是结果，关键是依赖方向和数据归属。

追问：是否要为每层建接口？答：只在存在替换、隔离副作用或测试价值时抽象。机械地为每个类加接口会增加跳转和维护成本。

09 HTTP 与 REST API

25 P0 POST 请求怎样实现幂等

考察点：是否会处理网络重试造成的重复副作用。

参考回答：客户端为一次业务操作生成 Idempotency Key，服务端把它与用户、接口和规范化请求摘要绑定。第一次请求创建处理中记录并执行；相同 key 和相同参数返回原结果，不同参数则拒绝。数据库唯一约束用于处理并发竞争，外部调用也要传递稳定业务单号。只在内存里查 key 无法跨进程或重启保证幂等。

追问：请求超时但服务端成功怎么办？答：客户端用同一 key 查询或重试，服务端返回已完成结果。不能新建业务单号再次执行。

26 P0 怎样设计统一错误响应

考察点：API 是否可调试又不泄露内部信息。

参考回答：响应应包含稳定错误码、面向调用方的说明、请求或 trace id，以及必要的字段级校验信息。HTTP 状态码表达协议层类别，业务码表达具体原因。内部堆栈、SQL 和密钥不能返回给客户端，但应与 trace id 关联写入日志。第三方错误要做归一化，并区分可重试、不可重试和需要人工处理。

追问：所有业务错误都返回 200 可以吗？答：会削弱网关、监控和客户端的标准行为。应合理使用 4xx 与 5xx，并在响应体补充业务码。

27 P1 API 分页 版本和鉴权怎样设计

考察点：能否兼顾演进与数据隔离。

参考回答：数据变化频繁时优先游标分页，并用稳定、唯一的组合排序键；小型后台可使用 offset，但要说明一致性和性能边界。破坏性变更通过版本或兼容字段演进，删除字段要有迁移窗口。鉴权负责确认身份，授权要在资源访问处校验归属和角色，不能只验证 JWT 合法就允许读取任意 ID。

追问：前端隐藏按钮算授权吗？答：不算。后端必须在每个敏感操作处做权限与资源归属检查。

10 PostgreSQL 与 SQLAlchemy

28 P0 一条查询变慢怎样定位

考察点：是否能从证据而不是猜测开始。

参考回答：先用 trace 或慢查询日志确认具体 SQL、参数范围和耗时分布，再看 EXPLAIN ANALYZE、实际行数、扫描方式、排序和回表情况。检查筛选、连接、排序是否有匹配索引，统计信息是否准确，是否出现 N 加 1 或一次读取过多列。索引会增加写入和存储成本，因此必须围绕真实访问模式建立，并在接近生产的数据量上验证。

追问：复合索引顺序怎么定？答：根据等值过滤、范围条件、排序和选择性综合决定，并用执行计划验证，不能只背最左前缀。

29 P0 事务隔离和行锁怎样回答

考察点：能否解释并发行为。

参考回答：PostgreSQL 默认 Read Committed，每条语句看到语句开始前已提交的数据。一个事务内两次查询可能得到不同结果。需要原子推进状态时，我会使用带旧状态条件的 UPDATE 并检查影响行数，或在确需读取后修改时使用 SELECT FOR UPDATE。更高隔离级别能减少异常，但也可能增加冲突和重试，不能替代业务幂等。

追问：死锁怎样处理？答：统一锁顺序、缩短事务、避免事务内远程调用，并捕获数据库死锁错误后对可重试操作做有限重试。

30 P1 SQLAlchemy Session 和连接池有哪些坑

考察点：是否理解 ORM 背后的资源。

参考回答：Session 是工作单元和身份映射边界，不应作为跨请求全局对象。请求结束必须提交或回滚并关闭。连接池大小要和 Worker 并发、数据库上限一起计算，多进程部署后总连接数会相乘。N 加 1、懒加载越过会话生命周期、长事务以及连接未释放都会造成问题。排查要同时看应用池等待、数据库活动连接和慢 SQL。

追问：池开得越大越好吗？答：不是。过大可能让数据库上下文切换和锁竞争更严重，应根据容量和压测结果设置。

11 Redis 一致性与并发控制

31 P0 你在项目里怎样使用 Redis

考察点：能否讲清数据性质与失效策略。

参考回答：简历中的场景包括 Celery Broker 或结果协作、会话和任务上下文、缓存以及验证码类短期数据。每种用途需要不同的 key 设计、TTL、序列化和故障策略。会话状态要考虑过期后如何恢复，缓存可以丢但要防止穿透和击穿，队列则要关注可靠投递和积压。面试时我会选一个自己真正实现过的链路展开，不把 Redis 的所有数据结构都说成用过。

追问：Redis 不可用怎么办？答：按用途决定降级。缓存可绕过并保护数据库；关键会话若无持久化来源就不能承诺无损恢复；任务队列故障需要暂停入口或切换恢复方案。

32 P0 Cache Aside 怎样保持一致

考察点：是否理解缓存只是副本。

参考回答：读时先查缓存，未命中读数据库并回填；写时先提交数据库，再删除缓存。删除失败要有重试或消息补偿，并设置合理 TTL。并发下仍可能读到旧值，强一致决策应直接读权威库或带版本号。热点 key 要防击穿，空结果可短暂缓存，但要避免恶意 key 造成穿透。

追问：为什么不是先删缓存再写库？答：两步之间若有并发读，旧值可能再次回填并长期存在。无论哪种顺序都有窗口，必须结合版本、补偿和业务容忍度设计。

33 P1 Redis 分布式锁怎样实现才安全

考察点：是否会识别锁过期与误删。

参考回答：获取锁使用带过期时间的原子 `set nx, value` 写唯一 token；释放时用 Lua 比较 token 后删除，避免删掉别人的锁。任务执行时间可能超过租期，需要评估续期或改用数据库状态机。即使锁实现正确，进程暂停或网络分区仍可能让两个执行者先后进入，所以关键写入还需要唯一约束、版本号或 fencing token 兜底。

追问：有锁就不用幂等了吗？答：不用。锁控制并发窗口，幂等保证重复请求或重试不会产生重复副作用，两者解决的问题不同。

12 Celery 与异步任务

34 P0 Celery 为什么会重复执行任务

考察点：是否理解消息确认和故障恢复。

参考回答：Worker 在执行中崩溃、确认丢失、可见性超时或调用方重试，都可能让同一消息再次被处理。工程上通常按至少一次语义设计，而不是假设恰好一次。任务要有稳定业务 key，写入用唯一约束或条件更新，外部副作用记录执行状态。确认时机、超时和重试参数要与任务耗时匹配。

追问：acks late 一定更可靠吗？答：它能让未完成任务重新投递，但也增加重复执行概率。任务必须幂等，并避免 Worker 被杀后留下不可恢复的外部副作用。

35 P0 Celery 重试怎样设计

考察点：能否区分瞬时与永久错误。

参考回答：网络超时、限流和部分 5xx 可以在总时限内指数退避并加入抖动；参数错误、权限错误和数据缺失通常不应盲目重试。每次失败记录分类、attempt 和下次时间，超过阈值进入失败队列或人工处理。重试不能重新生成业务标识，也不能在未知结果时重复调用非幂等第三方接口。

追问：第三方超时怎么办？答：先按业务单号查询结果；若对方支持幂等 key，用同一 key 重试；否则进入不确定状态并人工核对。

36 P1 长任务如何展示进度和支持取消

考察点：任务状态模型是否完整。

参考回答：创建任务时返回 task id，服务端持久化 queued、running、succeeded、failed、cancelling 和 cancelled 等状态，并记录版本与时间。Worker 分阶段更新可解释进度，客户端轮询或订阅事件。取消不是只改前端状态，Worker 要在安全检查点读取取消标志并停止后续副作用。结果和日志应有保留期限，状态更新用条件写避免旧 Worker 覆盖新状态。

追问：Celery revoke 能保证立刻停止吗？答：不能把它当成业务级强保证。已运行任务能否终止取决于执行池和信号处理，关键流程仍需协作式取消与状态校验。

13 Pytest 与自动化测试

37 P0 你会怎样测试一个新 API

考察点：测试是否覆盖边界与副作用。

参考回答：先为服务层写核心规则和错误路径测试，再用 API 测试验证请求校验、鉴权、状态码和响应 Schema。数据库操作使用隔离的测试库或事务夹具，第三方调用通过端口或 Client 接口替换。至少覆盖正常、空值、越权、重复提交、并发或超时中的关键风险。上线前再做一条包含数据库和队列的集成链路。

追问：覆盖率越高越好吗？答：覆盖率能发现未执行代码，但不能证明断言有效。优先覆盖高风险决策、失败路径和历史缺陷。

38 P1 FastAPI 异步测试怎么写

考察点：是否理解事件循环和应用生命周期。

参考回答：可使用 pytest 的 AnyIO 支持与 HTTPX AsyncClient 调用 ASGI 应用，在异步测试中 await 数据库或服务。若应用依赖 lifespan，需要显式触发生命周期。每个测试应隔离数据库状态，并避免在不同事件循环之间复用异步 Client。超时、取消和并发测试要使用可控的假服务，减少偶发等待。

追问：为什么测试偶尔出现 Task attached to a different loop？答：通常是异步对象在另一个事件循环创建并被复用，应在当前异步生命周期内创建和关闭。

39 P0 怎样测试 LLM 和第三方服务

考察点：能否在确定性测试与真实效果之间分层。

参考回答：单元测试不直接依赖真实模型，而是用固定响应验证 Prompt 组装、Schema 校验、重试、超时和错误映射。契约测试用录制或沙箱确认请求格式。离线评测使用固定样例集比较任务成功率、字段准确率、延迟和成本。少量端到端测试调用真实服务并设置预算，不能让每次提交都受模型随机性和网络影响。

追问：Mock 会不会掩盖问题？答：会，所以还需要契约测试和小规模真实调用。三层测试解决不同风险，不能只靠一种。

14 Debug 与可观测性

40 P0 接口突然变慢你怎样排查

考察点：定位顺序是否可执行。

参考回答：先确认影响范围、开始时间和 P50 P95 P99，而不是只看平均值。沿 trace 分解网关、应用、数据库、Redis、队列和第三方耗时，结合发布记录与资源指标找变化。对慢 SQL 看执行计划，对连接池看等待，对异步服务看事件循环阻塞。先采取限流、回滚或关闭非关键功能止损，再做根因修复和回归测试。

追问：没有 trace 怎么办？**答：**用请求 id 串联访问日志和下游日志，结合时间窗口做采样；修复后补齐埋点，避免下次仍靠猜。

41 P1 Python 进程内存持续增长怎样定位

考察点：是否理解对象保留与进程模型。

参考回答：先区分真正泄漏、缓存增长、任务积压和内存碎片。按时间观察 RSS、对象数量、请求类型和 Worker，使用 tracemalloc 比较快照，检查全局容器、未关闭响应、循环引用、日志缓冲和大对象。多进程下要分别看每个 Worker。可以设置 Worker 任务数或内存上限暂时止损，但最终要找到对象为何仍被引用。

追问：gc collect 能解决吗？**答：**只能回收不可达对象，仍被引用的数据不会释放。频繁强制 GC 还可能增加停顿。

42 P0 你做过哪些可观测性工作

考察点：简历内容是否真实可解释。

参考回答：慧学平台简历中写到 Prometheus 指标、psutil 和 tracemalloc 请求内存监控、慢请求堆栈日志，以及 Flask 和 SQLAlchemy 的 OpenTelemetry 链路追踪。我会准备一张实际指标或 trace 截图，说明埋点位置、标签控制和一次定位过程。高基数字段不能随意放入指标标签，敏感数据也不能直接进入日志。

追问：最关键的四类信号是什么？**答：**请求量、错误率、延迟和资源饱和度，再按数据库、队列和模型调用补充业务指标。

15 性能 稳定性与第三方依赖

43 P0 LLM 接口越来越慢怎样定位

考察点：能否分解模型链路。

参考回答：我会记录排队时间、连接时间、首 Token 延迟、生成时长、输入输出 Token、Provider、模型和重试次数。先判断是上下文变长、模型速度、限流重试、网络、连接池还是本地阻塞。优化可以包括裁剪上下文、缓存可复用结果、设置并发预算、流式返回和选择合适模型。不能只通过增加超时掩盖问题。

追问：为什么 P99 特别高？**答：**常见原因包括少量长上下文、重试放大、队列等待和第三方尾延迟，需要按维度拆分而不是看整体均值。

44 P0 怎样做背压和过载保护

考察点：服务是否会在高峰时自我拖垮。

参考回答：入口设置速率和并发上限，任务队列设置容量与优先级，数据库与外部 Client 的连接池也要有上限。超过容量时快速返回 429 或接受任务后异步处理，并给出重试时间。每一层的上限要协调，避免应用允许一千个请求同时等待只有二十个连接的数据库。监控队列长度和等待时间，必要时关闭非关键功能。

追问：为什么不能无限排队？**答：**队列会占用内存并让请求在过期后仍执行，最终增加尾延迟和恢复时间。

45 P1 第三方服务如何隔离故障

考察点：是否具备完整的韧性设计。

参考回答：每个第三方 Client 独立配置超时、并发、重试和熔断，错误统一分类。对读取操作可有限重试或使用短期缓存，对写操作必须依赖幂等与结果查询。降级要保留用户可理解的状态，例如稍后重试或人工处理。日志记录供应商、错误类别和 trace id，不记录密钥或完整隐私数据。

追问：熔断恢复怎么判断？**答：**经过冷却后允许少量探测请求，成功率稳定再逐步恢复，不能瞬间放开全部流量。

16 Docker 云平台与发布

46 P0 怎样制作 Python 服务镜像

考察点：是否理解可重复构建和运行安全。

参考回答：固定 Python 与依赖版本，使用多阶段构建或精简基础镜像，先复制锁文件安装依赖以利用缓存，再复制源码。使用非 root 用户，配置健康检查和只读或最小权限文件系统，密钥通过运行环境注入。镜像启动时不应动态下载依赖。数据库迁移与应用启动要明确顺序，并保留可回滚版本。

追问：一个容器里跑 Web 和 Celery 可以吗？**答：**开发环境可简化，生产通常分开部署，便于独立扩缩、重启和资源限制。

47 P0 没有 AWS 生产经验怎样回答

考察点：是否诚实，又能说明可迁移能力。

参考回答：我会明确说目前主要是 Docker Compose、Nginx、阿里云 OSS 和服务部署经验，没有把 EC2、ECS、RDS 说成独立生产维护过。然后说明我理解容器服务、对象存储、托管数据库、网络权限、健康检查和日志监控的共同模型，并给出学习计划。若面试官给出具体故障场景，我会先按通用层次分析，再确认 AWS 服务特有机制。

追问：怎样快速补齐？**答：**部署一个最小服务到测试账号，完成镜像、配置、RDS 迁移、日志、健康检查和回滚演练，并保留基础设施和验证记录。

48 P1 发布前后要检查什么

考察点：是否对最终交付结果负责。

参考回答：发布前确认构建产物、配置差异、数据库迁移兼容性、测试结果、容量和回滚方案。发布采用小流量或分批策略，观察错误率、延迟、资源、队列和关键业务指标。发布后执行冒烟测试并核对日志。若指标恶化，按阈值回滚，而不是边猜边继续扩大流量。变更记录应能关联提交、镜像和迁移版本。

追问：数据库迁移能回滚就够了吗？**答：**很多数据变更不可逆，应优先使用向前兼容的扩展迁移，代码稳定后再清理旧字段。

17 Coding Harness 工作方式

49 P0 你怎样使用 Claude Code Codex 或 Cursor 完成需求

考察点：是否有可重复的 AI 编程流程。

参考回答：我先写清目标、验收条件、禁止修改范围和测试命令，让工具读取相关入口、数据模型和现有测试。先要求它解释调用链并提出小步方案，再实现一组可审查的改动。完成后运行格式检查、类型检查、单元和集成测试，查看 diff 与日志，并手动验证关键路径。复杂任务分成多个提交，任何测试失败都保留证据后修复，不直接接受大段生成结果。

追问：什么时候不用 AI 编程工具？答：凭据处理、生产数据操作或安全边界不清时先人工确认；微小明确修改如果沟通成本更高也可直接完成。

50 P0 怎样给 Coding Agent 提供上下文

考察点：是否懂 Context Engineering。

参考回答：上下文应包含任务目标、接口契约、相关文件、领域约束、现有测试和可运行命令。优先让 Agent 用搜索定位，而不是塞入整个仓库。稳定规范放在仓库说明，当前决策放在任务提示，失败日志原样提供但要脱敏。若输出偏离，先检查缺少哪个事实或验收标准，再决定是否补上下文。

追问：为什么让 Agent 先解释代码？答：可以及早发现它读错入口或依赖，减少在错误假设上生成大范围修改。

51 P1 Harness 中哪些约束应自动化

考察点：能否把规范变成机器可执行门槛。

参考回答：格式、lint、类型、依赖漏洞、单测、API 契约和禁止路径修改适合自动化。数据库迁移可检查命名、顺序和兼容性；危险命令和生产凭据由权限系统限制。架构取舍、业务含义和高风险发布仍需人工审查。自动门槛要快速、可重复，并提供清楚错误，否则只会诱导绕过。

追问：测试全绿就能合并吗？答：不能。还要审查需求是否满足、测试是否有效、权限和数据边界是否被破坏，以及是否引入不可接受的运维成本。

18 AI 生成代码的验证与责任

52 P0 怎样验证 AI 生成的代码

考察点：是否真正理解并承担结果。

参考回答：我先检查改动是否只覆盖需求范围，再沿数据流审查输入、权限、事务、异常和资源释放。然后运行 lint、类型检查和测试，补充模型没有写到的边界用例，并在本地或测试环境复现成功与失败路径。依赖和配置变更要核对官方文档。最后阅读完整 diff，删除无关重构、重复逻辑和看似合理但没有证据的注释。

追问：最常见的问题是什么？**答：**错误假设现有接口、吞掉异常、异步函数里调用阻塞库、测试只验证 happy path，以及为解决局部问题引入过度抽象。

53 P0 AI 写出的代码出了 Bug 谁负责

考察点：责任意识。

参考回答：提交和交付的人负责。AI 工具不承担需求理解、数据安全、上线审批或事故响应责任。我会把它当作能快速提出实现的协作者，但必须由我验证。若发生问题，先止损和复盘测试与审查为什么没有发现，再改进 Harness，而不是把原因归结为模型偶尔出错。

追问：还会继续用吗？**答：**会，但要根据缺陷类型增加契约、测试或权限约束，让同类错误更早暴露。

54 P1 怎样让 AI 帮你 Debug 和重构

考察点：是否能利用工具而不丢失判断。

参考回答：Debug 时提供最小复现、错误堆栈、相关日志和最近变更，请它列出可证伪假设与验证命令；逐个验证，不直接接受第一结论。重构前先锁定行为测试，规定不改变的接口和性能边界，再按小步修改。对于并发、事务和安全代码，我会手动推演失败顺序，并增加相应测试。

追问：如何避免 Agent 自己写测试证明自己正确？**答：**验收条件由需求和历史行为决定，关键测试由人审查，必要时让独立评审流程只看需求与 diff。

19 LLM API 与结构化输出

55 P0 Structured Output 为什么比提示返回 JSON 更可靠

考察点：是否理解协议约束与业务校验的层次。

参考回答：普通提示只能要求模型尽量输出 JSON，仍可能出现多余文本、缺字段或类型错误。支持 Schema 的 Structured Output 能在模型或客户端层约束结构，Pydantic 再做运行时校验。但结构合法不等于业务正确，例如工具名称、资源权限、金额范围和状态都要额外校验。对不支持原生 Schema 的 Provider，可以使用解析、有限纠错和明确失败策略。

追问：校验失败是否无限让模型修复？答：不能。修复次数要受 deadline 和成本预算限制，持续失败应返回可诊断错误或降级到人工处理。

56 P0 多个 LLM Provider 怎样适配

考察点：是否能处理兼容接口背后的差异。

参考回答：先定义内部请求、消息、工具、流事件和错误模型，各 Provider Adapter 负责参数映射、鉴权、流解析与错误归一化。能力表记录是否支持工具、结构化输出、思考参数和上下文长度。兼容 OpenAI 协议不代表语义完全一致，参数降级只能针对已知错误并记录，不能把所有 400 都静默重试。切换后还要用固定评测集验证效果。

追问：如何避免 Provider 配置在并发请求间串掉？答：请求开始时创建不可变配置快照并显式传递，不依赖可变全局变量或未清理的线程本地状态。

57 P1 怎样控制模型调用成本

考察点：能否建立质量与成本的量化关系。

参考回答：按任务记录输入输出 Token、重试、模型、缓存命中和成功率。简单分类或字段提取使用更轻模型，复杂规划和代码审查再用强模型。裁剪无关历史、检索必要证据、限制输出长度，并缓存不含用户隐私且规则版本稳定的结果。成本优化不能只看单价，还要看失败重试、人工接管和延迟。

追问：缓存模型回答有什么风险？答：用户权限、规则版本和数据时效可能不同，key 必须包含相关维度，关键交易结果通常不缓存。

20 Agent Tool Calling 与安全

58 P0 Tool Calling 的完整链路是什么

考察点：是否把工具调用理解为受控执行协议。

参考回答：服务端向模型提供名称、说明和 JSON Schema；模型返回调用意图和参数；程序校验工具是否允许、参数是否合法、用户是否有权限以及当前状态是否可执行；通过后调用真实服务，把结构化结果返回模型或直接生成业务响应。每一步记录 trace 和状态。高风险操作需要确认或审批，模型不能绕过服务端规则。

追问：工具描述写得很详细就安全吗？答：不安全。描述属于软约束，真正的权限、参数和副作用控制必须在执行器中完成。

59 P0 Agent workflow 怎样支持失败恢复

考察点：状态持久化和副作用边界。

参考回答：把节点、输入、输出、attempt 和状态持久化，区分可重试计算与不可重复副作用。每个有副作用的工具使用幂等 key，并在调用前后记录业务状态。Worker 重启后从已确认的 checkpoint 恢复，不能仅靠对话文本猜测进度。超过重试阈值后进入人工处理，同时保留原始错误和上下文。

追问：是不是所有节点都要持久化？答：不一定。昂贵、外部副作用或需要人工中断的节点优先；纯函数式且成本低的步骤可重算。

60 P1 Prompt Injection 怎样防

考察点：是否理解 RAG 内容与用户输入都不可信。

参考回答：系统指令不能成为唯一防线。检索文档和网页按不可信数据处理，不允许其中内容改变工具权限。工具使用最小权限和白名单，参数做 Schema 与业务校验，敏感动作需要确认。输出进入 HTML、SQL 或 shell 时使用对应安全编码或参数化接口。日志记录被拒绝的调用，但避免泄露系统提示和凭据。

追问：让模型先判断内容是否恶意够吗？答：不够。模型判断可以作为信号，硬权限和数据边界必须由程序执行。

21 RAG Context Engineering 与评测

61 P0 讲一下你的文献 RAG 链路

考察点：简历中的技术是否能连成完整流程。

参考回答：我先用 Dify 验证知识库和工作流，再用 LangChain 独立实现解析、索引、检索、重排和生成。Grobid 把论文解析成结构化 XML，BGE M3 生成向量表示，ChromaDB 管理向量检索，同时融合关键词召回，再用 reranker 对候选证据排序并压缩上下文。回答时保留来源和片段边界，避免把检索到的文字直接当成必然正确的结论。

追问：BGE M3 和 reranker 是同一个东西吗？**答：**不是。embedding 模型负责召回表示，reranker 对问题与候选片段做更精细相关性判断。

62 P0 准确率提升约 23% 和 Token 减少 21% 怎样证明

考察点：指标是否有可复现口径。

参考回答：面试前必须准备样本数、问题来源、人工标注标准、Dify 基线配置、LangChain 配置、模型版本和每条请求 Token 日志。要说明 23% 是相对提升还是百分点，21% 是否包含输入、输出和失败重试。若多个环节同时改变，只能说明整条链路相对基线改善，不能把结果归因于某一个框架。无法核实的数字应主动修正。

追问：给一个计算例子？**答：**若基线 60%，改进后 73.8%，相对提升是 23%，百分点增加 13.8；这只是计算示例，不是项目原始记录。

63 P1 你怎样做 LLM Evaluation

考察点：是否从 Demo 走向可持续优化。

参考回答：先定义任务成功条件，例如字段准确率、工具选择正确率、最终任务完成率、引用支持率、延迟和成本。固定样例集按正常、边界、攻击和历史失败分层，离线比较模型、Prompt 和检索版本；上线后抽样人工复核并监控拒答、重试和人工接管。LLM Judge 可辅助扩展，但要用人工样本校准偏差。

追问：Prompt 改好一个例子就能上线吗？**答：**不能。它可能只对单例过拟合，需要对完整回归集比较并保留版本和回滚能力。

22 Agent 实习项目深挖

64 P0 介绍自动化测试 Agent

考察点：能否讲清目标、工具和结果验证。

参考回答：实习项目把自然语言安全测试需求映射为漏洞扫描、报告解析和结果分析工具链。我的讲法会按输入、任务路由、工具参数、执行状态、结果解析和最终报告展开。模型负责理解需求与选择步骤，扫描器负责执行，服务端验证参数并保存状态。扫描结果只是工具证据，最终报告还要关联原始输出、过滤误报并说明限制。

追问：你个人负责哪一段？**答：**按真实提交说明自己实现或参与的路由、工具封装、状态管理和测试，不把团队全部能力归为个人完成。

65 P0 LangGraph 中怎样做路由和重试

考察点：是否理解图状态而非只会调用库。

参考回答：状态中保存目标、已完成步骤、工具结果、错误类别和重试次数；条件边根据结果决定继续、回退、人工确认或结束。重试只回到可安全重放的节点，并在状态中增加 `attempt`。工具参数错误可回到模型纠正，外部服务瞬时错误走退避，权限错误直接终止。图结构要能解释为什么走到当前节点。

追问：多 Agent 有什么必要？**答：**只有角色、上下文或工具权限确实需要隔离时才拆分。简单任务用单 workflow 更易调试。

66 P1 短期记忆和长期记忆怎样设计

考察点：是否能控制上下文规模和隐私。

参考回答：短期记忆保存当前任务需要的消息、工具结果和状态，随会话过期；长期记忆只保存经筛选、可复用且允许持久化的信息，并带来源、更新时间和用户范围。长对话通过摘要压缩，但重要业务字段应结构化保存，不能只留模型摘要。用户应能查看或删除长期信息，检索时必须按身份过滤。

追问：摘要错了怎么办？**答：**保留关键原始事件或可追溯引用，对摘要做版本化；关键决策重新读取权威数据。

23 慧学教育平台深挖

67 P0 多租户部署怎样隔离

考察点：简历中的多租户是否有具体边界。

参考回答：简历描述为每租户独立 PostgreSQL 库和 Celery 队列，通过 Docker Compose 编排。这样数据库和任务积压的故障域更清楚，但配置、连接数、迁移和监控数量会增加。应用选择租户后必须从可信身份或部署配置获取数据源，不能接受用户随意指定库名。公共组件和对象存储同样需要租户前缀与权限控制。

追问：如何批量迁移几十个租户库？**答：**记录每库版本，分批执行、失败隔离和重试，迁移脚本保持向前兼容，并提供进度与告警。

68 P0 画像增量计算怎样保证正确

考察点：能否解释脏标记和异步重算。

参考回答：业务数据变化时只标记相关学生或班级及数据版本为脏，任务按稳定 key 合并重复请求，读取截止版本的数据计算，并用条件更新写回结果。若计算期间又发生变化，版本检查会保留脏状态并再次调度，避免旧结果覆盖新数据。任务要幂等，失败记录原因与重试次数。

追问：为什么不每次同步计算？**答：**画像计算可能耗时且会增加接口尾延迟，异步可以削峰，但需要接受短暂最终一致并向调用方暴露数据更新时间。

69 P1 Gunicorn 与 SQLAlchemy 连接池怎样配合

考察点：是否理解多进程资源乘法。

参考回答：Gunicorn 每个 Worker 是独立进程，各自拥有连接池。数据库总连接需求大致受 Worker 数、每池大小和溢出连接共同影响，不能只看单进程配置。预加载应用时要避免在 fork 前创建不可安全继承的连接。通过池等待时间、数据库活动连接和请求延迟压测，选择 Worker 与池大小。

追问：CPU 核数乘二加一是固定公式吗？**答：**不是。它只是经验起点，I/O 比例、内存、异步模型和数据库容量都会改变最佳值。

24 Homegrown 的可迁移能力

70 P0 Java 项目能为 Python 岗位证明什么

考察点：能否提炼跨语言能力而不回避差异。

参考回答：homegrown 能证明我做过完整产品和复杂后端边界，包括业务状态机、PostgreSQL 持久化、Redis 会话、SSE、JWT、文件处理和多 Provider 模型适配。这些问题迁移到 Python 后仍需要同样的事务、幂等、权限和可观测性设计。但它不能直接证明我的 Python 语法和工具链熟练度，所以我会用慧学平台、Agent 实习以及现场代码来完成这部分证明。

追问：如果用 FastAPI 重写你会怎么拆？**答：**先保持领域和数据契约，用路由、服务、Repository 和 Provider Adapter 分层；索引任务拆到 Celery，不做一次性全量重写。

71 P1 SSE 和任务取消怎样迁移到 Python

考察点：能否把经验落到异步实现。

参考回答：FastAPI 可以用异步生成器持续产生事件，代理层要关闭不必要缓冲并发送心跳。客户端断开时取消生成任务，协程在 finally 中释放流和并发许可。下游模型如果不支持真正取消，也要停止继续读取并记录状态。事件格式、部分分片和多行 data 需要解析测试，不能只在浏览器手工验证。

追问：为什么不用 WebSocket？**答：**如果主要是服务端单向流，SSE 更简单且能复用 HTTP；需要高频双向交互时再考虑 WebSocket。

72 P0 怎样解释简历与当前 homegrown 代码的差异

考察点：诚信与版本意识。

参考回答：简历概括了项目演进能力，但面试时应按可展示版本说明。当前工作区最能完整证明的是资料结构化、学习状态、模型适配和 SSE；向量表与索引定义不能单独证明完整 pgvector 检索主链已经运行，配置中的 Skill 数量也要按实际文件核对，简化复习规则不能称为完整 FSRS。我会准备对应提交或明确说属于历史版本和下一步计划。

追问：为什么要主动说限制？**答：**面试官通常会沿代码追问。清楚区分当前实现和设计方案，反而能证明我理解系统边界。

25 现场 Python SQL 与代码题

Python 题 带并发上限的调用

题目：并发调用一组异步接口，最多同时五个，单项超时后记录失败，并保证许可释放。

```
async def call_one(item, client, limiter):
    async with limiter:
        try:
            async with asyncio.timeout(3):
                return await client.fetch(item)
        except TimeoutError:
            return {"item": item, "status": "timeout"}
```

答案：Semaphore 形成并发上限，async with 保证退出时释放许可，timeout 控制单项时间。生产代码还要处理取消、错误分类、trace id 与总 deadline，不能无限创建 Task。

SQL 题 游标分页

题目：查询某租户已完成任务，按 created_at 和 id 倒序，每页二十条。

```
SELECT id, created_at, status
FROM task
WHERE tenant_id = :tenant_id AND status = 'SUCCEEDED'
AND (created_at, id) < (:last_time, :last_id)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

答案：第一页省略游标条件；联合索引可围绕 tenant_id、status、created_at、id 设计并用执行计划验证。id 打破相同时间的排序并列。游标分页减少大 offset 扫描，但不是跨时刻一致快照。

代码题 LRU 与 TopK

LRU 可用 dict 加双向链表实现，访问或更新后移动到头部，超容量淘汰尾部，平均 $O(1)$ ；说明结构默认非线程安全。TopK 可维护大小为 K 的最小堆，时间 $O(n \log K)$ ，空间 $O(K)$ 。现场写完要主动覆盖 K 小于等于零、K 大于数据量、重复值和空输入。

SQL 题 条件更新实现状态推进

```
UPDATE task SET status = 'RUNNING', version = version + 1
WHERE id = :id AND status = 'QUEUED' AND version = :version;
```

答案：只有影响一行才获得推进资格。外部调用放在可控事务边界之外，并以稳定业务 key 做幂等；不能持有数据库事务等待长时间模型调用。

26 系统设计 AI 代码问题处理服务

设计题

设计一个服务：用户提交 GitHub Issue，系统读取指定仓库上下文，由 Coding Agent 给出补丁，在隔离环境运行测试，人工确认后合并。

回答框架

入口服务验证用户与仓库权限，创建不可变任务和幂等 key。任务表记录 QUEUED、ANALYZING、PATCHED、TESTING、WAITING_REVIEW、APPROVED、FAILED 和 CANCELLED，并使用版本号做条件更新。API 只负责创建与查询，长任务进入队列。

Worker 在隔离工作区检出固定提交，给 Agent 最小文件与仓库规则。Agent 只能在授权路径编辑，不能访问生产凭据。补丁生成后运行 Ruff、类型检查、单元测试、集成测试和依赖安全检查。测试日志、diff、模型版本、Prompt 摘要和 Token 成本作为产物持久化。

人工审查页面展示需求、基线提交、补丁、测试和风险。只有具备权限的用户可以批准。批准后再次确认分支未漂移；若基线变化则重新应用和测试。合并操作使用幂等 key，失败进入可恢复状态，不让模型直接操作主分支。

面试官可能追问

怎样防止恶意仓库指令影响 Agent？把仓库内容作为不可信输入，工具权限与系统规则由外层执行器控制。

如何隔离代码执行？使用短生命周期容器、无默认网络、只读基础镜像、CPU 内存时间限制和临时凭据。

怎样评测系统？统计补丁通过测试率、人工采纳率、回滚率、首次结果延迟、总成本和不同缺陷类型的成功率。

怎样恢复？任务每一步写 checkpoint，副作用幂等；Worker 重启后从已确认状态继续，未知外部结果先查询再重试。

这是一道拟议系统设计题。回答时不要把其中的隔离、评测或 GitHub 自动合并说成你的既有上线成果。

27 行为面试 模拟流程与准备清单

73 P0 讲一次你定位复杂问题的经历

考察点：是否有真实行动和证据。

参考回答：选择慧学平台的慢请求、内存或连接池问题，按背景、影响、观察证据、假设、验证、修复和结果回答。重点说明你先看到了什么指标或日志，如何排除错误方向，改了哪段代码，怎样通过压测或回归测试确认。若没有可量化结果，就说现象消失、错误不再复现或监控恢复，不编造百分比。

追问：如果最后是同事找到原因？答：如实说明自己的排查范围、提供的证据和共同决策，不把团队成果改写成个人独立完成。

74 P0 你怎样与产品和前端协作

考察点：能否把模糊需求变成可验收接口。

参考回答：先确认用户动作、输入、成功状态、失败状态和权限，再用接口示例或 OpenAPI 对齐字段、状态码和异步进度。对模型结果不稳定的功能，提前约定加载、重试、人工修正和降级体验。开发中用小版本联调，变更记录兼容影响；上线前共同走一遍正常、空数据、超时和重复点击。

追问：需求临时变化怎么办？答：先说明对数据、接口和工期的影响，给出最小可交付方案和后续版本，而不是默默扩大范围。

75 P0 你的到岗和实习安排是什么

考察点：实习岗位的基本稳定性。

参考回答：面试前填写真实的最早到岗日期、每周可出勤天数、可持续月份和学校安排。如果存在论文、课程或住宿冲突，应提前说明确定边界和解决方式。不能只说时间灵活，也不要为了通过面试承诺无法履行的出勤计划。

追问：能接受高强度迭代吗？答：可以说明自己能配合合理的发布节奏，同时关注需求优先级、测试和恢复方案；高强度不等于省略质量门槛。

一轮 50 分钟模拟面试

0 至 5 分钟做自我介绍与求职动机。5 至 18 分钟深挖慧学平台，覆盖多租户、Celery、连接池和可观测性。18 至 28 分钟考 Python 异步、FastAPI、Pydantic 与 pytest。28 至 38 分钟问 Coding Harness、结构化输出和 Tool Calling。38 至 45 分钟完成一题代码或 SQL。45 至 50 分钟反问团队。

可以反问面试官

团队当前最希望实习生解决的是 Python 业务服务、Agent 编排，还是 Coding Harness 基础设施。一个需求从 Issue 到上线有哪些测试和评审门槛。现有 LLM 应用最主要的问题是成功率、延迟、成本还是可观测性。实习生第一个月的交付目标和反馈方式是什么。

面试前两天

第一天校准项目事实，准备慧学平台和 Agent 实习各一条完整调用链，以及 RAG 指标原始记录。第二天完成一个 FastAPI 小项目，至少包含 Pydantic v2、异步 Client、pytest、Ruff、Docker 和一条 CI；录一遍 50 分钟模拟面试并删掉无法举证的表述。

28 资料来源与证据定位

岗位与简历

岗位依据为用户提供的 Python 后端实习生招聘页面截图，页面显示公司为上海夯智交互文化科技，岗位强调 Python Web 服务、PostgreSQL、Redis、异步任务、测试 Debug、Docker 与云平台、LLM API、Structured Output、Tool Calling、RAG、Agent、Coding Harness，以及对 AI 生成结果负责。

简历依据为全栈与 Agent 应用简历 V4。个人经历、职责与指标应由候选人结合源代码、提交记录和运行材料复核。

官方技术资料

[S1] Python asyncio Tasks, 用于核对任务取消、TaskGroup 和 timeout。

<https://docs.python.org/3.12/library/asyncio-task.html>

[S2] FastAPI Concurrency and async await, 用于核对 async def 与阻塞调用边界。

<https://fastapi.tiangolo.com/async/>

[S3] FastAPI Async Tests, 用于核对 pytest AnyIO 与 HTTPX 测试。

<https://fastapi.tiangolo.com/advanced/async-tests/>

[S4] FastAPI Deployment Concepts, 用于核对 Worker 进程与复制。

<https://fastapi.tiangolo.com/deployment/concepts/>

[S5] Pydantic Models, 用于核对模型校验、strict mode、extra 与 model construct。

<https://docs.pydantic.dev/latest/concepts/models/>

[S6] Celery Tasks, 用于核对任务确认、幂等和重试语义。

<https://docs.celeryq.dev/en/stable/userguide/tasks.html>

[S7] pytest Fixtures, 用于核对测试夹具与隔离。<https://docs.pytest.org/en/stable/how-to/fixtures.html>

[S8] PostgreSQL Transaction Isolation, 用于核对 Read Committed 和并发语义。

<https://www.postgresql.org/docs/current/transaction-iso.html>

[S9] Model Context Protocol Server Concepts, 用于核对 tools resources prompts 的服务端能力。

<https://modelcontextprotocol.io/docs/learn/server-concepts>

[S10] OpenAI Structured Outputs, 用于核对 JSON Schema 约束能力。

<https://platform.openai.com/docs/guides/structured-outputs>

项目证据准备

慧学平台：准备 Flask 路由与鉴权入口、SQLAlchemy 会话和连接池配置、Celery 任务与脏标记逻辑、多租户部署文件、Prometheus 或 OpenTelemetry 截图，以及一次真实故障记录。

Agent 实习：准备可脱敏的状态结构、路由图、工具 Schema、失败分类与重试逻辑，明确个人贡献与团队边界。

文献 RAG：准备评测集、Dify 基线、LangChain 配置、逐题结果、Token 日志、Grobid 输出样例及混合检索和 rerank 代码。

homegrown：当前工作区可重点展示模型适配、结构化输出、SSE、学习状态和 Redis 会话；涉及 pgvector、Skill 数量与 FSRS 时按具体版本回答。

最后检查

把所有括号内待填信息换成真实值。删除无法解释的数字和工具名。为每个主项目准备一张架构图、一条成功链、一条失败链和一个测试。现场回答先讲结论，再讲证据和边界。